

CSS 핵심 개념과 선택자, 박스모델

👉 오늘의 강의 학습 요약

본 수업은 웹의 클라이언트/서버 실행 구조를 정리한 뒤, HTML이 브라우저에서 DOM 트리로 객체화되어 렌더링된다는 관점을 기반으로 CSS의 역할을 연결했습니다. 이어서 CSS 문법(선택자, 중괄호, 프로퍼티/값, 주석)과 적용 방식(인라인/인터널/익스터널), 적용 우선순위 및 캐스케이딩(상속)과 명시도(우선순위 점수)를 통해 "어떤 노드에 어떤 규칙이 최종 적용되는지"를 체계적으로 설명했습니다.

이후에는 선택자를 단순 선택(전체/태그/아이디/클래스)에서 결합(AND/OR, 자손/자식, 인접/일반 형제), 속성 선택자(일치/시작/끝/포함)와 의사 클래스, 의사 요소로 확장했으며, 박스 모델(콘텐츠, 패딩, 보더, 마진)과 box-sizing, 마진 축약/상쇄, 중앙정렬(margin:auto)까지 다뤘습니다. 또한 display/visibility, position(Static/Relative/Fixed/Absolute/Sticky)와 z-index, float, 배경 관련 속성, 크기 단위(px, %, vw/vh, em/rem), max-width, 색상 표현, opacity, 폰트 패밀리(제네릭/폰트)까지 실무에 필요한 CSS 기반기를 폭넓게 정리했습니다.

1. 웹 실행 구조와 DOM 개요

웹 컴포넌트는 정적(HTML/CSS/JS)과 동적(JSP/Servlet)로 구분되며, HTML은 서버에 저장되더라도 실행은 브라우저에서 이루어지고 브라우저는 HTML을 DOM 트리로 객체화해 렌더링합니다.

클라이언트(브라우저)와 서버 환경을 구분하는 것이 출발점입니다. 서버에는 HTML, JSP, Servlet 같은 “요청 대상 파일”이 존재하며, 요청/응답 과정에서 어디에서 실행되느냐에 따라 정적/동적으로 나뉩니다.

구분	대표 예	실행 위치	응답 형태	핵심 특징
정적 컴포넌트	HTML, CSS, JavaScript	클라이언트 (브라우저)	파일 그대로 다운로드	서버는 별도 실행 없이 전달하고 브라우저가 해석, 실행합니다.
동적 컴포넌트	JSP, Servlet	서버	실행 결과로 생성된 HTML	서버가 실행하여 결과 HTML을 만들어 브라우저에 응답합니다.

서버에서 실행되는 프로그램은 서버사이드, 브라우저에서 실행되는 것은 클라이언트사이드로 부릅니다. 이후 React, Vue 등도 기본적으로 클라이언트에서 실행되는 흐름으로 이해하면 정리됩니다.

HTML 문서는 브라우저 내부에서 태그마다 대응되는 객체가 생성되고, 이 객체들이 트리 구조로 메모리에 올라갑니다. 이 구조가 DOM 트리이며, DOM은 Document Object Model로 “문서를 객체로 모델링하여 관리하는 방식”을 의미합니다.

DOM 트리가 만들어지면 특정 노드(태그)를 선택해 스타일(CSS)을 적용하거나, 값을 조회, 변경, 수정, 삭제하는 작업이 가능해집니다. CSS 관점에서의 핵심은 “원하는 노드를 정확히 찾는 선택자”이며, DOM 트리를 그리며 탐색하면 선택자 해석이 쉬워집니다.

HTML 태그는 화면에 렌더링될 때 박스 형태로 취급되며, 배치 방식이 인라인/블록 레벨로 구분됩니다. 인라인은 콘텐츠만큼만 너비를 가지므로 가로로 이어질 수 있고, 블록은 기본적으로 가로 너비 전체를 차지해 다음 요소가 줄바꿈됩니다.

인라인/블록의 중요한 차이는 width/height 적용 여부입니다. 인라인 요소는 규칙상 콘텐츠 크기를 따르기 때문에 width/height를 지정해도 적용되지 않고, 블록 요소는 지정이 적용됩니다.

핵심 키워드

#정적 컴포넌트

#동적 컴포넌트

#클라이언트사이드

#서버사이드

#DOM 트리

2. 폼 전송과 쿼리스트링 기본

사용자 입력 요소는 form으로 감싸야 전송되며, action으로 서버의 타겟을 지정하고 method(GET/POST)에 따라 쿼리스트링이 URL에 노출되거나 숨겨져 전달됩니다.

사용자 입력 화면(체크박스, 라디오, 텍스트 입력 등)을 서버로 전송하려면 form 태그로 입력 요소들을 감싸야 합니다. form이 없으면 전송 버튼을 눌러도 서버로 값이 넘어가지 않습니다.

form의 action은 “어떤 서버 자원(HTML/JSP/Servlet)이 받을 것인지”를 지정합니다. 실무에서는 주로 서버 실행이 가능한 Servlet 같은 동적 자원이 수신자가 됩니다.

method는 GET/POST 두 가지가 핵심이며, 지정하지 않으면 기본은 GET입니다. GET은 쿼리스트링이 URL에 노출되어 전달되고, POST는 URL에는 노출되지 않지만 쿼리스트링 형태의 데이터가 본문으로 전달됩니다.

쿼리스트링은 키=값 구조이며, 키는 입력 태그의 name, 값은 사용자가 입력한 값입니다. 여러 값이 전달되면 &로 구분됩니다.

핵심 키워드

#form

#action

#method

#GET

#쿼리스트링

3. CSS 문법과 적용 방식, 우선순위

CSS는 선택자로 DOM 노드를 지정한 뒤 { 프로퍼티: 값; } 형태로 스타일을 적용하며, 적용 방식은 인라인/인터널/익스터널 3가지이고 기본 우선순위는 인라인 > 인터널 > 익스터널입니다.

CSS는 HTML을 “예쁘게 스타일링”하는 언어이며 정식 명칭은 Cascading Style Sheets이고, 수업에서는 CSS3를 표준으로 사용합니다. 표준안 관점에서 HTML은 HTML5, CSS는 CSS3, JavaScript는 ECMAScript 표준 흐름과 연결됩니다.

기본 문법은 선택자 다음에 중괄호를 열고, 프로퍼티 : 값; 을 작성합니다. 여러 선언은 세미콜론으로 구분합니다. CSS 주석은 자바의 멀티라인 주석과 동일하게 /* ... */ 입니다.

```
h1 {  
  color: red;  
  font-size: 20px;  
}
```

CSS 적용 방식은 3가지입니다.

방식	위치	특징	대표 태그/속성
인라인 (Inline)	시작 태그의 style 속성	요소에 가장 가깝고 우선순위가 가장 높습니다.	style="color:red;"
인터널 (Internal)	HTML <head> 내부 <style>	문서 단위로 적용되며 외부 파일 없이 관리합니다.	<style>...</style>
익스터널 (External)	별도 .css 파일	재사용과 유지보수에 유리하며 링크로 불러옵니다.	<link rel="stylesheet" href="...">

같은 요소에 대해 충돌이 발생하면 기본 우선순위는 인라인 > 인터널 > 익스터널입니다. “요소에 가까운 선언이 이긴다”로 이해하면 빠르게 정리됩니다.

또한 클래스 선택자는 . (점), 아이디 선택자는 # (해시)로 표현하며, 대소문자 구분은 일반적으로 하지 않지만 관례적으로 소문자를 사용합니다. 인라인 스타일은 가능하나 HTML과 CSS 분리가 깨져 중복, 유지보수 문제가 생기기므로 권장되지 않습니다.

핵심 키워드

#CSS3

#선택자

#인라인

#인터널

#익스터널

4. 캐스케이딩과 명시도(Specificity)

캐스케이딩은 부모에 준 스타일이 자식에게 상속되는 성질이며, 명시도는 선언 순서가 아니라 선택자 우선순위 점수로 최종 스타일이 결정되는 규칙입니다.

캐스케이딩(Cascading)은 상위 요소에 적용한 스타일이 하위 요소(자식/자손)까지 “폭포처럼” 내려가는 개념입니다. 특정 요소에 직접 색을 준 적이 없는데 색이 적용됐다면, 조상(부모/상위)에서 상속된 경우를 먼저 의심해야 합니다.

자식은 상속된 스타일을 그대로 따르되, 자신에게 다시 스타일을 선언해 “재정의(override)”할 수 있습니다. 즉, 캐스케이딩은 상속을 기본으로 하면서도 재정의 가능성이 핵심입니다.

명시도는 CSS가 “마지막 선언이 무조건 이긴다”가 아니라는 점을 설명합니다. 선택자마다 우선순위가 있고, 우선순위가 높은 선택자가 최종 적용됩니다.

우선순위(높음→낮음)	예시	개념
인라인 스타일	<code>style="..."</code>	가장 강력하지만 유지보수에 불리합니다.
아이디 선택자	<code>#id</code>	문서 내 유일해야 하며 명시도가 높습니다.
클래스/속성 선택자	<code>.class</code> , <code>[attr]</code>	재사용이 잦고 실무에서 핵심입니다.
태그 선택자	<code>div</code> , <code>p</code>	범위가 넓고 명시도가 낮습니다.

선택자를 조합하면 명시도가 합산되어 더 강해집니다. 예를 들어 `#id` (100)과 `h1` (001)을 조합하면 101이 되어, 단순 태그 선택자보다 훨씬 우선됩니다.

핵심 키워드

#캐스케이딩

#상속

#명시도

#우선순위

#재정의

5. 기본 선택자와 결합 연산(AND/OR)

CSS 선택자는 전체/태그/아이디/클래스로 노드를 찾고, 공백, 꺾쇠, 붙여쓰기, 쉼표를 통해 자손/자식/AND/OR 의미로 결합할 수 있습니다.

전체 선택자 `*` 는 DOM 트리의 모든 노드를 대상으로 하며, 범위가 가장 넓습니다. 태그 선택자는 동일 태그를 전부 선택하므로 범위 제어가 필요합니다.

아이디 선택자 `#id` 는 유일해야 하며, 클래스 선택자 `.class` 는 중복 사용이 가능하고 가장 많이 사용됩니다. 클래스는 한 요소에 여러 값을 공백으로 나열할 수 있으며, 각 값을 개별 클래스로 인식합니다.

선택자 결합에서 혼동하기 쉬운 연산자 의미를 명확히 구분해야 합니다.

표기	의미	예시 해석
공백	자손(자식 포함)	<code>.select .aa</code> 는 <code>.select</code> 의 자손 중 <code>.aa</code> 입니다.
<code>></code>	자식(1레벨)	<code>#one > h1</code> 은 <code>#one</code> 의 “직계 자식” h1입니다.
붙여쓰기	AND 조건	<code>.select.aa</code> 는 두 클래스를 동시에 가진 요소입니다.
<code>,</code>	OR 조건	<code>.select, .aa</code> 는 둘 중 하나만 만족해도 선택됩니다.

이 결합은 DOM 트리를 그려보면 빠르게 이해됩니다. 특히 공백(자손)과 붙여쓰기(AND), 쉼표(OR)가 시각적으로 비슷해 오해하기 쉬우므로, “DOM 구조 + 조건 해석” 순서로 판단하는 습관이 중요합니다.

핵심 키워드 `#전체 선택자` `#아이디 선택자` `#클래스 선택자` `#자식 선택자` `#자손 선택자`

6. 계층(형제) 선택자와 속성 선택자

형제 선택자는 `+` 로 인접 형제 1개, `\~` 로 뒤쪽 형제 전부를 선택하며, 속성 선택자는 `[]` 로 속성명/값/패턴(시작, 끝, 포함) 기반 탐색을 지원합니다.

계층 구조는 자식(1레벨), 자손(2레벨 이상), 부모(1레벨 위), 조상(그 이상), 형제(같은 레벨) 개념을 함께 이해해야 합니다. 형제는 영어로 sibling으로도 표현되며, 문서에서 sibling이 나오면 “같은 레벨”을 떠올리면 됩니다.

형제 선택자는 두 가지가 핵심입니다.

- `A + B` 는 A의 바로 뒤(인접)의 형제 B 하나만 선택합니다.
- `A \~ B` 는 A 뒤에 나오는 형제 B를 모두 선택합니다.

속성 선택자는 대괄호 `[]` 를 사용합니다. 아이디는 보통 `#id` 로 찾지만, 속성 관점에서는 `[id]` 처럼 “id 속성이 존재하는 요소”로도 찾을 수 있음을 확인했습니다.

속성 값 매칭은 정확 일치뿐 아니라 패턴 기반도 지원합니다.

문법	의미	예시
<code>[attr]</code>	속성 존재	<code>[id]</code> 는 id 속성이 있는 요소입니다.
<code>[attr="v"]</code>	정확 일치	<code>[id="two"]</code> 는 id가 two인 요소입니다.
<code>[attr^="v"]</code>	v로 시작	<code>[href^="https"]</code> 는 https로 시작합니다.
<code>[attr\$="v"]</code>	v로 끝	<code>[href\$="4"]</code> 는 4로 끝납니다.
<code>[attr*="v"]</code>	v 포함	<code>[href*="man"]</code> 은 man을 포함합니다.

속성 선택자 앞에 태그 등을 붙이면 AND 조건이 되어 범위를 제한할 수 있습니다. 예를 들어 `a[id]` 는 “a 태그이면서 id 속성이 있는 요소”로 좁혀집니다.

핵심 키워드

#인접 형제 선택자

#일반 형제 선택자

#속성 선택자

#시작 일치

#포함 일치

7. 의사 클래스, 의사 요소와 상태 기반 스타일

의사 클래스는 `:` 로 상태를 기준으로 요소를 선택하고, 의사 요소는 `::` 로 요소의 일부(첫 글자, 첫 줄, 앞/뒤 삽입 등)를 꾸미는 방식입니다.

의사 클래스(pseudo-class)는 콜론 `:` 으로 시작하며, 상태나 구조를 기준으로 요소를 선택합니다. 특히 체크박스/라디오에서 “체크된 항목만 선택”은 일반 선택자만으로 어렵기 때문에 `:checked` 는 실무에서 매우 중요합니다.

구조 기반 의사 클래스는 `:first-child`, `:last-child`, `:only-child`, `:nth-child(n)` 등을 다룹니다. 여기서 중요한 해석 규칙은 “선택된 요소 자체의 자식”이 아니라, 부모를 기준으로 **n번째 자식인지**를 판단한다는 점입니다.

또한 `:not(...)` 을 이용하면 특정 조건을 부정하여 “첫 번째가 아닌 요소”처럼 선택 범위를 만들 수 있습니다. `:empty` 는 자식이 없는 요소를 선택합니다.

입력 폼에서는 `:enabled`, `:disabled`, `:focus` 등을 통해 활성화/비활성 및 포커스 상태에 따라 UI를 제어할 수 있습니다. `:disabled` 는 속성 단독 표기와 키/값 표기 모두 가능하다는 점도 함께 정리되었습니다.

의사 요소(pseudo-element)는 콜론 두 개 `::` 로 시작하며, 요소의 “일부분”을 스타일링합니다. 대표적으로 `::first-letter`, `::first-line` 이 있으며, `::before`, `::after` 는 content 프로퍼티와 함께 앞/뒤에 텍스트나 이미지를 삽입할 수 있습니다.

`::selection` 은 사용자가 드래그로 선택한 영역의 스타일을 바꾸며, `::placeholder` 는 입력창의 안내 문구(placeholder)의 스타일을 지정합니다. `::backdrop` 은 모달(dialog) 배경을 스타일링하는 데 활용됩니다.

핵심 키워드

`#:checked`

`#:nth-child`

`#:not`

`#:before`

`#:placeholder`

8. 박스 모델과 box-sizing, 여백 규칙

모든 요소는 콘텐츠, 패딩, 보더, 마진으로 구성된 박스이며, 기본 마진(예: body 8px)이 존재할 수 있고 box-sizing을 border-box로 두면 크기 계산이 직관적입니다.

박스 모델은 콘텐츠(content)를 중심으로 안쪽 여백(padding), 테두리(border), 바깥 여백(margin)이 둘러싼 구조입니다. 개발자 도구(F12)에서도 박스 모델을 시각적으로 확인할 수 있으며, 기본적으로 브라우저가 제공하는 기본 마진(예: body 8px)을 인지해야 레이아웃 오차를 줄일 수 있습니다.

레이아웃을 예측 가능하게 만들기 위해 전체 선택자에 margin을 0으로 초기화하고 시작하는 패턴을 자주 사용합니다. 또한 여백과 테두리는 위/오른쪽/아래/왼쪽 4방향이 공통이며, 시계 방향 규칙이 반복됩니다.

width/height는 “콘텐츠의 크기”로 이해하기 쉽지만, 실제 렌더링 크기에는 padding/border/margin이 개입할 수 있어 눈에 보이는 크기가 달라질 수 있습니다. 이 문제를 단순화하는 핵심이 box-sizing 입니다.

box-sizing 값	의미	실무 사용
content-box	width/height가 콘텐츠만 의미합니다.	기본값이지만 계산이 번거롭습니다.
border-box	width/height에 padding/border를 포함합니다.	레이아웃 계산이 쉬워 자주 사용됩니다.

마진/패딩은 축약 표기가 가능하며, 4개/3개/2개/1개 표기는 모두 “시계 방향(위-오-아-왼)” 규칙을 따릅니다. 마진은 바깥 여백이며 margin: auto 로 블록 요소의 중앙 정렬을 구현할 수 있습니다.

마진 상쇄(margin collapsing)는 인접한 블록 요소의 위/아래 마진이 단순 합산(예: 20+30=50)되지 않고 일부 상쇄되어 더 작은 값으로 적용되는 현상입니다. 개발자 도구로 실제 적용 값을 확인하며 이해하는 방식이 권장됩니다.

핵심 키워드

#박스 모델

#padding

#border

#margin

#box-sizing

9. border, radius, display/visibility, float

border는 두께, 스타일, 색상을 축약해 자주 지정하고 **border-radius**로 모서리를 둥글게 하며, **display/visibility**는 보이기/영역 유지가 다르고 **float**은 콘텐츠 흐름에 맞춘 배치를 만듭니다.

border는 테두리이며, 두께(**width**), 스타일(**style**), 색상(**color**)을 각각 지정하거나 축약형으로 한 줄에 작성하는 방식이 가장 많이 쓰입니다. **border** 스타일은 **solid**(실선), **dotted**(점선) 등으로 표현됩니다.

border-radius는 모서리를 둥글게 하며 값이 클수록 더 둥글어집니다. 50%를 주면 원형에 가까운 형태가 만들어질 수 있습니다.

display는 요소의 렌더링 방식을 바꿉니다. 블록/인라인 변환이 가능하며, `display: none`은 요소를 “DOM에서 제거된 것처럼” 영역 자체가 사라집니다. 반면 `visibility: hidden`은 보이지 않지만 영역은 유지됩니다.

인라인 요소는 **width/height**가 적용되지 않는 제약이 있으므로, “가로 배치(인라인) 특성을 유지하면서도 크기를 주고 싶을 때” `display: inline-block`을 사용합니다.

float은 요소를 “띄워서” 텍스트 흐름과 함께 자연스럽게 배치하는 데 활용됩니다. 이미지와 문단을 같은 라인 흐름으로 배치할 때 **float left/right**가 대표적입니다.

핵심 키워드

`#border-radius`

`#display`

`#visibility`

`#inline-block`

`#float`

10. position, z-index와 레이어링

position은 top/right/bottom/left로 위치를 지정하며 **static**은 위치 지정이 먹지 않고, **relative**는 원래 위치 기준, **fixed**는 뷰포트 기준, **absolute**는 부모(**relative** 유무)에 따라 기준이 달라지고 **sticky**는 임계점 전후로 동작이 바뀝니다.

position은 요소를 특정 위치에 배치하기 위한 속성입니다. 위치를 정하려면 top/right/bottom/left 중 보통 2개를 조합해 좌표를 만듭니다.

position의 핵심 타입은 다음과 같이 정리됩니다.

position	기준	특징
static	문서 흐름	기본값이며 top/left 등이 적용되지 않습니다.
relative	원래 배치 위치	static으로 배치되었을 위치를 기준으로 상대 이동합니다.
fixed	뷰포트(화면)	스크롤/화면 크기 변화에도 고정 위치를 유지합니다.
absolute	부모 또는 뷰포트	부모가 relative 등 위치 컨텍스트를 가지면 부모 기준, 아니면 뷰포트 기준입니다.
sticky	혼합	평소에는 static처럼 흐르다가 임계점에 도달하면 fixed처럼 고정됩니다.

absolute가 헷갈리는 이유는 기준이 두 가지가 될 수 있기 때문입니다. 자식이 **absolute**일 때 부모에 **relative**가 있으면 부모 박스를 기준으로 위치를 잡고, 부모에 위치 지정이 없으면 화면(뷰포트)을 기준으로 잡습니다.

z-index는 포지션으로 인해 요소가 겹칠 때 “앞/뒤” 레이어 순서를 결정합니다. 값이 작을수록 아래로, 클수록 위로 올라오며 음수도 사용할 수 있습니다.

핵심 키워드

#position

#relative

#absolute

#fixed

#z-index

11. 배경 속성, 크기 단위, 색상, 투명도

`background`는 색/이미지/반복/크기/위치/attachment로 구성되며, 단위는 px(절대)과 %(부모 기준), vw/vh(뷰포트 기준), em/rem(폰트 기준)을 구분하고, 색상은 키워드, 16진수, rgb(a)로 표현하며 `opacity`로 투명도를 제어합니다.

`background-color`는 배경색, `background-image`는 배경 이미지를 지정합니다. 이미지는 반복될 수 있으며 `repeat`, `repeat-x`, `repeat-y`, `no-repeat`로 제어합니다. `background-size`는 px로 지정하거나 `contain` 등으로 화면에 맞게 조절할 수 있습니다.

`background-position`은 좌표(예: `left bottom`) 또는 픽셀/비율로 위치를 조정합니다. `background-attachment`는 스크롤에 따라 배경이 함께 움직일지(scroll) 고정될지(fixed)를 결정합니다.

크기 단위는 절대/상대로 나뉘며, 절대는 px를 중심으로 이해합니다. 상대 단위는 %와 vw/vh, 그리고 글꼴 기반의 em/rem이 핵심입니다. %는 보통 부모 크기를 기준으로 계산되며, vw/vh는 뷰포트(화면) 기준의 비율입니다.

`max-width`는 % 기반으로 요소가 커질 때 "무한정 커지는 것"을 제한하기 위한 속성입니다. 특히 반응형에서 이미지가 과도하게 커지는 문제를 막기 위해 자주 사용됩니다.

색상 표현은 키워드(red 등), 16진수(#RRGGBB, #RGB), rgb(0~255), rgba(투명도 포함) 방식이 있습니다. 투명도는 rgba의 a(0~1)로 주거나, 요소 전체에 `opacity`로 적용할 수 있습니다.

`visibility`는 `visible`/`hidden`/`collapse` 등을 가지며, `collapse`는 주로 테이블에서 영역까지 접는 목적으로 사용된다는 점을 함께 다뤘습니다.

핵심 키워드

#background

#vw/vh

#max-width

#rgba

#opacity

12. 폰트 패밀리(제네릭/폰트)와 폰트 지정 규칙

글꼴은 제네릭 패밀리(카테고리)와 폰트 패밀리(실제 글꼴명)로 나뉘며, `font-family`는 실제 글꼴들을 나열한 뒤 마지막에 제네릭 패밀리를 두어 폴백을 구성하는 것이 관례입니다.

글꼴은 카테고리(제네릭 패밀리)와 실제 글꼴명(폰트 패밀리)을 구분해야 합니다. 대표 제네릭 패밀리는 serif(꼬리 있음)과 sans-serif(꼬리 없음)이며, 실제 글꼴로는 Times New Roman, Georgia, Consolas 등 다양한 폰트가 존재합니다.

브라우저(예: Chrome) 설정의 글꼴 맞춤 설정에서 제네릭 패밀리별로 어떤 실제 폰트를 사용할지 매핑되어 있습니다. CSS에서 폰트를 지정하지 않으면 브라우저의 기본(표준) 글꼴 설정에 따라 렌더링됩니다.

`font-family`는 여러 글꼴을 나열하는데, 앞의 글꼴이 없으면 다음 글꼴로 넘어가며, 마지막에는 제네릭 패밀리를 두어 최소한의 폴백을 확보하는 것이 일반적인 작성 규칙입니다.

작성 패턴	의도	효과
<code>font-family: "맑은 고딕", "굴림", sans-serif;</code>	특정 폰트를 우선 적용하고 없으면 대체합니다.	환경 차이(폰트 설치 여부)에도 안정적입니다.
<code>font-family: sans-serif;</code>	카테고리만 지정합니다.	브라우저의 sans-serif 매핑 폰트가 적용됩니다.
미지정	브라우저 기본 사용	표준 글꼴(브라우저 설정)이 적용됩니다.

또한 폰트 크기 단위에서 `em`은 부모 기준으로 누적되어 중첩 구조에서 의도치 않게 커질 수 있으므로, 실무에서는 `rem` 사용을 권장한다는 흐름까지 연결했습니다.

핵심 키워드

`#font-family`

`#제네릭 패밀리`

`#serif`

`#sans-serif`

`#rem`